TESTE DE SOFTWARE

Refatoração de Testes e Detecção de Test Smells

Análise e refatoração da suíte Jest de UserService

Luidi Cadete Silva
Matrícula: 1447551

18 de outubro de 2026

Repositório de entrega
github.com/LuidiC/test-smelly

1. Análise dos Test Smells

O serviço gerencia usuários em memória. A análise considera o arquivo original **userService.smelly.test.js**; os números abaixo correspondem às suas linhas reais. O arquivo foi movido de **test/** para **__tests__/**, conforme o enunciado, com conteúdo idêntico, verificado por SHA-256. A lógica de negócio foi preservada.

1.1 Eager Test - linhas 18-31

O caso “deve criar e buscar um usuário corretamente” executa criação e consulta, intercalando ações e asserções. Uma falha inicial pode impedir a avaliação da busca, dificultando o diagnóstico e a manutenção. A correção separa as responsabilidades: criar é a ação de um caso e a preparação do caso de consulta.

1.2 Lógica condicional no teste - linhas 33-52

O **for** da linha 40 e o **if/else** da linha 42 controlam quais asserções serão executadas para usuários comuns e administradores. O leitor precisa interpretar o fluxo para descobrir os cenários. A versão limpa usa dois casos explícitos, verificando tanto o retorno da desativação quanto o estado persistido do usuário.

1.3 Teste frágil por formatação - linhas 54-63

O relatório exige uma linha completa com vírgulas e quebra de linha, além do cabeçalho exato. Uma mudança visual pode quebrar o teste sem alterar o comportamento. Bob é cadastrado, mas não verificado. A refatoração confere nomes e IDs de ambos e os estados em cenários próprios, sem fixar cabeçalho, ordem ou pontuação.

1.4 Falso positivo em try/catch - linhas 66-75

O único **expect** está no **catch**. Se a validação de idade for removida, a chamada não lança e o teste termina com sucesso sem verificar nada. Isso oculta uma violação da regra de maioridade. A correção usa **toThrow**, exigindo que a exceção ocorra, e acrescenta um caso válido para exatamente 18 anos.

1.5 Teste ignorado e vazio - linhas 77-79

O **test.skip** não contém asserções e não executa o cenário sem usuários. Seu nome menciona uma lista, mas a API disponível retorna relatório textual. O novo arquivo verifica a mensagem de ausência de usuários. O skip permanece somente no original, em respeito à exigência de preservação.

A análise manual e o lint são complementares: a ferramenta identificou asserções condicionais e teste ignorado/vazio, mas não diagnosticou o Eager Test ou o acoplamento à formatação do relatório.

2. Processo de refatoração

O teste de idade foi escolhido porque poderia aprovar uma implementação defeituosa. Os dois trechos abaixo são extraídos diretamente dos arquivos do projeto.

ANTES - original, linhas 66-75

```
test('deve falhar ao criar usuário menor de idade', () => {
  // Este teste não falha se a exceção NÃO for lançada.
  // Ele só passa se o `catch` for executado. Se a lógica de validação
  // for removida, o teste passa silenciosamente, escondendo um bug.
  try {
    userService.createUser('Menor', 'menor@email.com', 17);
  } catch (e) {
    expect(e.message).toBe('O usuário deve ser maior de idade.');
  }
});
```

DEPOIS - suíte limpa, linhas 61-70

```
test('rejeita um usuário menor de idade', () => {
  // Arrange
  const dados = { nome: 'Menor', email: 'menor@email.com', idade: 17 };

  // Act
  const criarMenor = () => userService.createUser(dados.nome, dados.email, dados.idade);

  // Assert
  expect(criarMenor).toThrow('O usuário deve ser maior de idade.');
});
```

Arrange: os dados do menor ficam explícitos no cenário. **Act:** a função adia a operação para que o matcher possa observar sua execução. **Assert: toThrow** executa essa função e exige a exceção esperada; a ausência de erro passa a causar falha. O teste deixa de depender de um catch e passa a verificar diretamente a regra de negócio.

Nos demais casos, criação e consulta foram separadas; usuários comuns e administradores ganharam cenários independentes; o relatório passou a conferir conteúdo relevante. O **beforeEach** limpa o banco em memória, evitando dependência de ordem. Os 18 testes novos não usam if, for ou try/catch nos seus corpos.

Verificação da capacidade de detectar defeitos

Em cópias temporárias, seis alterações deliberadas foram testadas: idade mínima, limite de 18 anos, bloqueio de administrador, mudança de status, presença de Bob no relatório e campos obrigatórios. A suíte limpa detectou **6/6**; a original, **2/6**. A remoção da idade mínima escapou da original, confirmando o falso positivo. Esse experimento dirigido não é um escore completo de mutação.

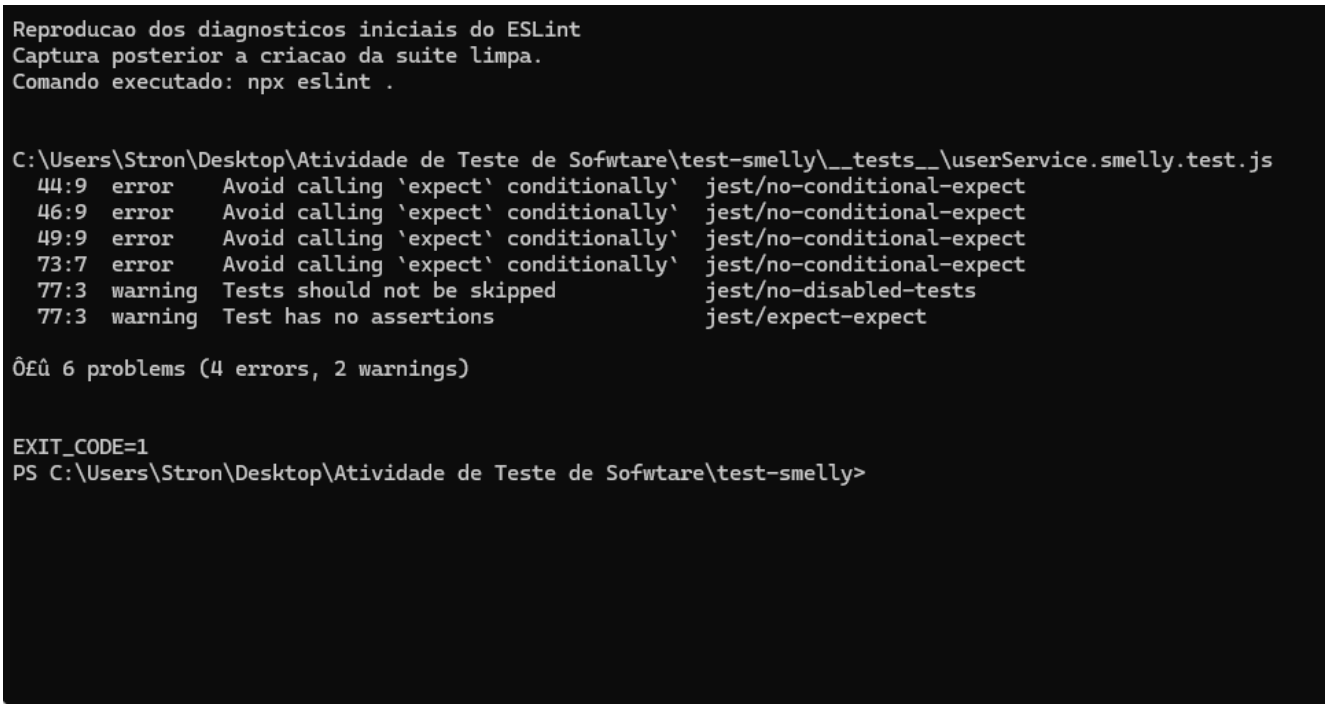
Configuração da ferramenta

Foram utilizados Node 24.14.1, npm 11.11.0, Jest 29.7.0, ESLint 8.57.1 e eslint-plugin-jest 28.14.0. O ESLint 8 foi fixado para aceitar o **.eslintrc.json** exigido; é uma versão fora de suporte, escolhida por compatibilidade com o enunciado. Os presets recomendados e as três regras obrigatórias estão configurados.

3. Ferramenta, validação e conclusão

Captura real do ESLint

Figura 1. Captura manual de **npx eslint .**: reprodução dos diagnósticos iniciais, realizada após a criação da suíte limpa. Os 4 erros e 2 avisos permanecem exclusivamente no original preservado. Arquivo: docs/evidencias/eslint-inicial.png.



Os erros das linhas 44, 46 e 49 apontam asserções no if/else; o da linha 73 aponta a asserção no catch. Na linha 77, há avisos de teste ignorado e sem asserções. A regra de títulos idênticos está ativa, mas não encontrou ocorrência.

Validação	Resultado registrado
Jest original	4 aprovados; 1 ignorado
Jest completo (npm test)	22 aprovados; 1 ignorado original; 0 falhas
ESLint da suíte limpa	0 erros; 0 avisos; código de saída 0
ESLint global final	4 erros; 2 avisos, somente no original
Cobertura da suíte limpa	100%: instruções, ramificações, funções e linhas

O diagnóstico global permanece visível porque o original foi preservado; a melhoria é comprovada pelo arquivo limpo sem diagnósticos. O único teste ignorado é original. Uma restrição de cache foi resolvida com acesso normal, sem alterar código; os logs foram preservados.

Conclusão

Testes limpos tornam o comportamento esperado explícito e as falhas mais fáceis de diagnosticar. A refatoração eliminou fluxos condicionais problemáticos, reduziu acoplamento à apresentação e corrigiu uma validação que passava sem asserções. O ESLint automatiza parte desse controle e ajuda a sustentar a qualidade do projeto; análise manual e execução continuam necessárias. Cobertura total não prova ausência de defeitos, por isso também foram verificadas alterações deliberadas.

Fontes: enunciado oficial, pp. 1-4; CleitonSilvaT/test-smelly; eslint-plugin-jest; guia de migração do ESLint. Saídas completas e auditoria: docs/evidencias/ e docs/analise.md.